CHAPTER 03

Cloud-native application architecture

For Business and Technical Analysts in international commercial banking

Antonio Piemontese

October 2026



CHAPTER 03

Learning objectives

01

Explain cloud-native capabilities

Connect automation, elasticity and recovery to business needs.

02

Compare platform responsibilities

Identify the control and operating work behind each compute option.

03

Trace state through change

Explain what survives instance replacement, scaling and a new release.

04

Justify a proposal with evidence

Define service outcomes, capacity limits and useful validation tests.

What cloud-native means

Cloud-native applications use programmable platforms to support frequent change, elastic capacity and recovery.

These practices can apply in public, private and hybrid clouds.

Automation

Describe and repeat deployment and operations.

Replaceable compute

Keep durable business state outside instances.

Platform capabilities

Use services with explicit contracts and limits.

Observable behaviour

Measure outcomes and investigate failures.

PLATFORM CHOICES

Compute choices and responsibility

Option	Useful control	Operating question
Virtual machines	OS and runtime configuration	Who patches and scales the instances?
Managed application platform	Application behaviour and settings	Does it support our background work?
Managed Kubernetes	Workload and orchestration policies	Who owns cluster and workload operations?
Functions	Event or request handlers	What are the concurrency and execution limits?

Choose the least platform complexity that meets the workload and bank requirements.

HYPOTHETICAL BANKING CASE

Banking pilot: cloud deployment brief

Business need

One team serves existing customers in one pilot country. Application receipt must remain reliable while assessment continues separately.

Working assumptions

Approved managed platform available.

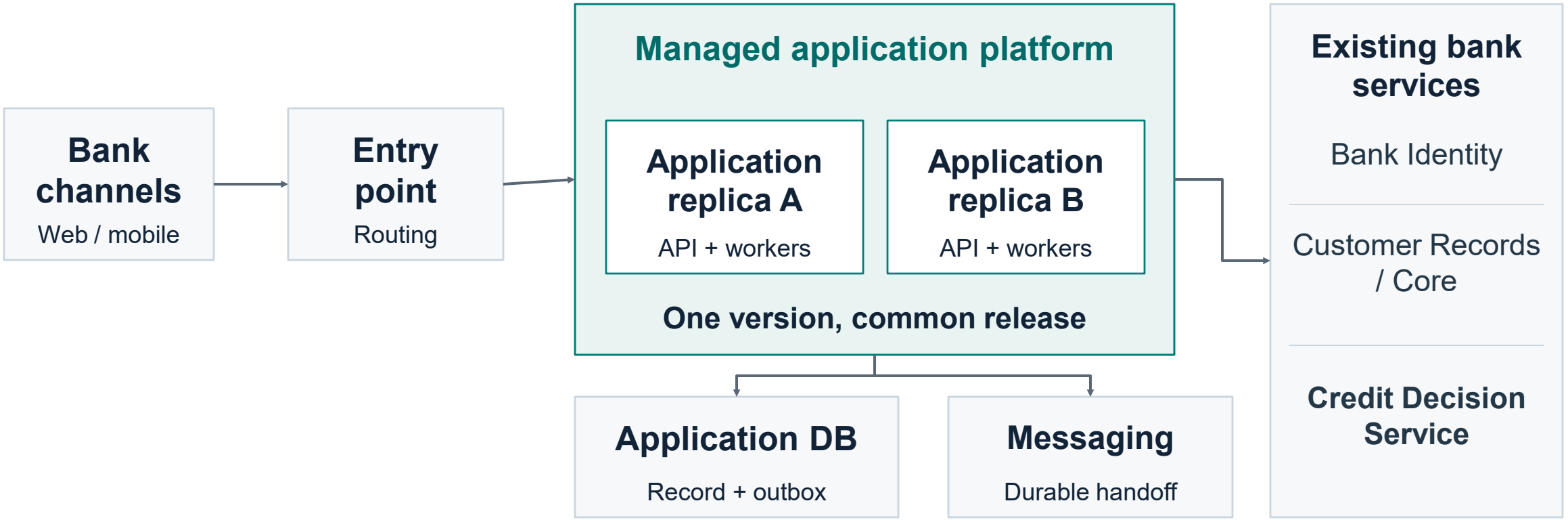
Credit Decision Service owns approval.

Approved bank connectivity is feasible.

Baseline target	p95 receipt ≤ 2 s at 20 valid submissions/s. Healthy intake/storage. Commit before receipt.

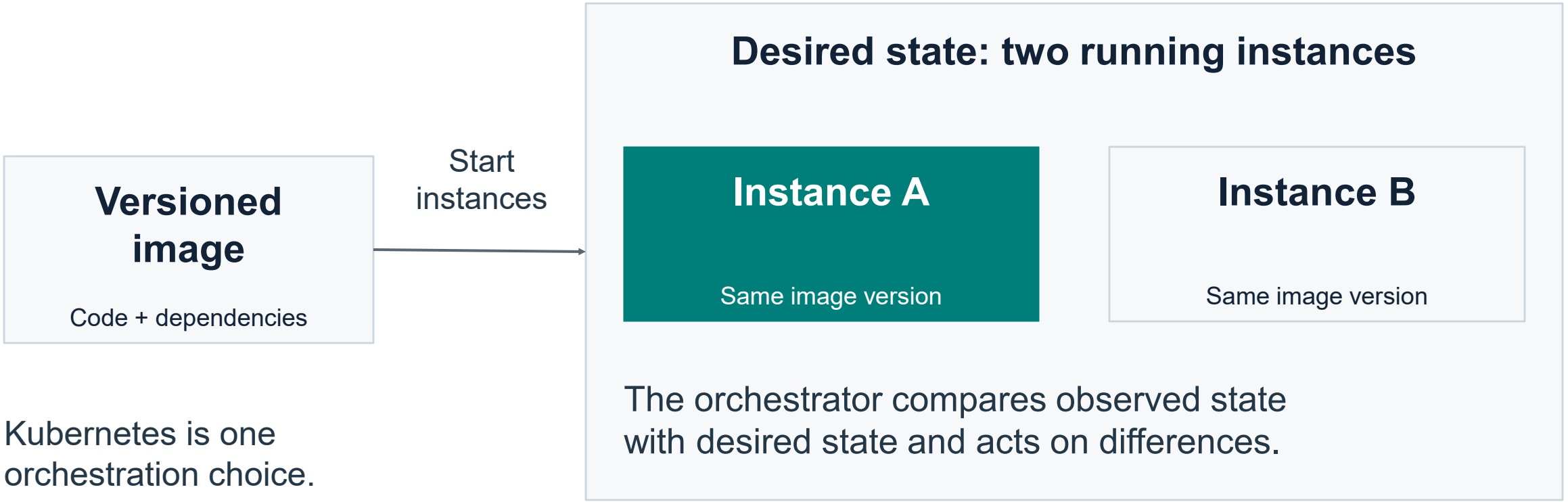
HYPOTHETICAL ARCHITECTURE

A cloud-native architecture for the pilot



Arrows show dependencies. The existing Credit Decision Service remains the authority for assessment and approval.

Containers and orchestration

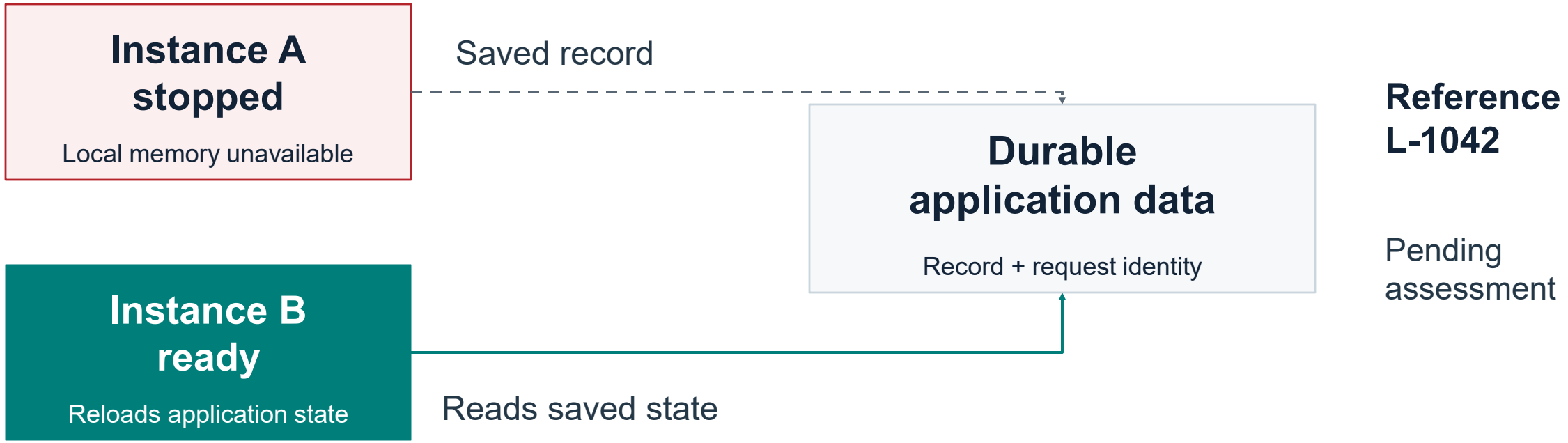


Kubernetes is one orchestration choice.

An image supplies a runnable package. The platform still needs capacity, isolation and recovery controls.

Stateless compute and durable data

Durable records let a healthy instance continue the workflow.



Dashed path: a record saved before replacement. Solid path: a ready instance reads the saved state.

Configuration, secrets and identity

Environment configuration

Supply endpoints and settings outside the build.

Use the same artifact with different configuration.

Validate and audit important configuration changes.

Secrets and workload identity

Keep credentials out of source code and images.

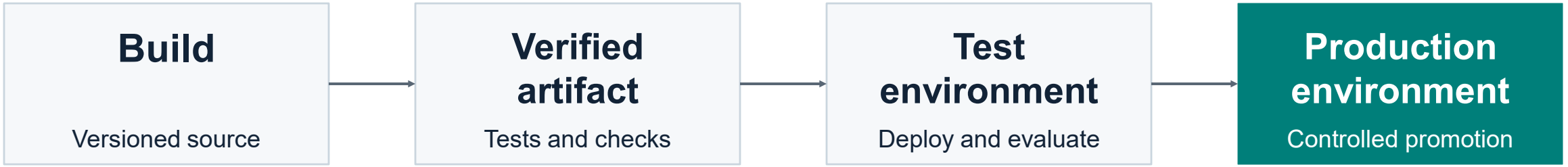
Use scoped workload identity where supported.

Protect secret access and test credential rotation.

An identity establishes who is calling. Authorisation determines what it may do.

AUTOMATED DELIVERY

Build and artifact promotion



Trace code, tests and approval evidence to a specific artifact version.

Use the same artifact digest. Supply configuration for each environment.

Review question Can we identify exactly what ran and restore a compatible earlier version?

KUBERNETES EXAMPLE

Health checks and safe replacement

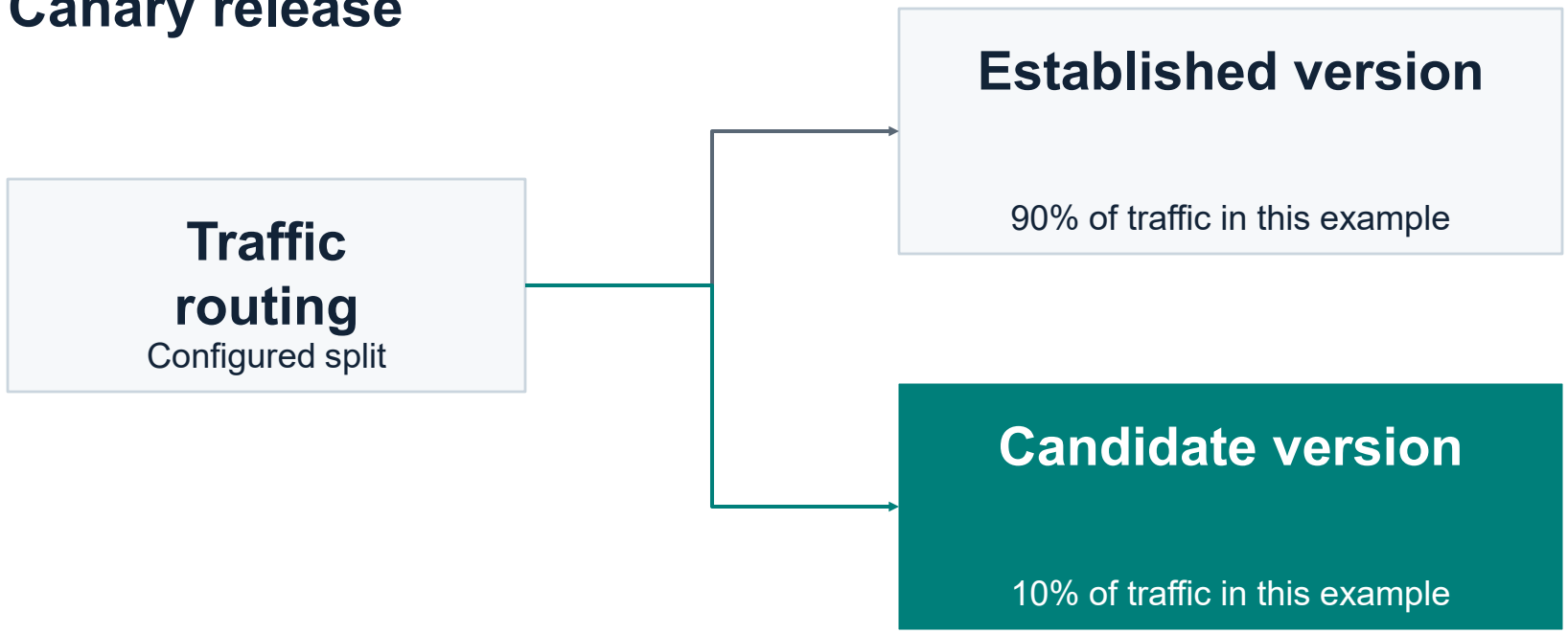
Probe	Question	Effect
Startup	Has the application started?	Allows initialisation before other probes run
Readiness	Can this instance accept traffic?	Controls eligibility for matching Service traffic
Liveness	Can the process make progress?	Can trigger a container restart under policy

On shutdown, stop new work and complete or safely resume work already accepted.

CONTROLLED CHANGE

Safe rollout and rollback

Canary release



Observe outcomes

Errors, receipt latency and the age of pending work.

Decision limits

Stop or reverse the rollout when agreed limits fail.

Illustrative traffic split. Rollback requires compatible interfaces and data changes.

ELASTICITY

Autoscaling and capacity limits

Signal	Possible indication	Capacity check
CPU utilisation	Compute pressure	Other resources may limit throughput
Queue age	Work waits longer	Downstream concurrency and backlog limits
Receipt latency	Customer-visible slowdown	Find the bottleneck before adding instances

Scaling takes time. Bound instance count, database connections and dependency calls.

OPERATIONAL EVIDENCE

Observability and service objectives

Observability links customer outcomes to the behaviour of the application and its dependencies.

Track receipt latency, successful acknowledgements and the age of pending work.

Metrics

Rates, durations and resource use.

Logs

Events with context and no sensitive payloads.

Traces

Work across calls and asynchronous handoffs.

Indicators and objectives

SLI: measure. SLO: target over a time window.

Resilience and security boundaries

Failure and recovery

Spread capacity across required failure domains.

Test replacement and dependency failures.

Verify backup and restore for durable state.

Security responsibilities

Apply scoped identity and approved network paths.

Control data location, access and retention.

Verify artifacts and audit privileged actions.

Name each owner and check the guarantees of the selected services.

Cost, portability and operating effort

Cost control

Measure service use and idle capacity.

Include storage, network and telemetry costs.

Budget for support and recovery work.

Portability

Images standardise application packaging.

Service APIs and data create migration work.

Test an exit path for important dependencies.

Evaluate total operating effort alongside the service bill.

HYPOTHETICAL DECISION RECORD

Pilot platform decision

ADR-003 Application execution platform

Status: Proposed

Context

One team delivers the pilot. The scenario assumes an approved managed platform and bank connectivity.

Decision

Use a managed application platform with durable backing services and a common application release.

Consequences

Verify background work, connectivity and recovery.

Revisit when evidence fails or the need for platform control changes.

GROUP EXERCISE

12 MINUTES

Exercise: a peak during replacement

Hypothetical campaign peak

Submissions rise from 20 to 80 per second for five minutes.

One application instance is replaced. The credit service keeps its agreed capacity limit.

Your output

01 Locate the durable state boundary.

02 Set scaling and lifecycle controls.

03 Define peak admission and customer status.

04 Specify evidence and a recovery owner.

Baseline: p95 receipt ≤ 2 s at 20 valid submissions/s, with healthy intake/storage.

Define and test peak behaviour at 80 submissions/s.

EXERCISE DEBRIEF

Debrief: elasticity with boundaries

01 **Durable state**

Keep records outside instances and preserve request identity.

02 **Bounded capacity**

Scale within dependency limits and define overload admission.

03 **Safe replacement**

Stop new work, drain requests and resume safely after failure.

04 **Customer outcome and evidence**

Keep “Pending assessment” truthful. Test baseline and peak behaviour.

REFERENCE MATERIAL

References and further reading

Cloud-native principles and application design

Cloud Native Computing Foundation and the Twelve-Factor App

- CNCF cloud-native definition Twelve-Factor App

Runtime operations and observability

Kubernetes and OpenTelemetry

- Kubernetes concepts OpenTelemetry signals

Architecture choices and shared responsibility

Microsoft Architecture Center and Amazon Web Services

- Architecture design principles Shared responsibility model

Primary references. Topic-specific links and teaching context appear in the speaker notes.